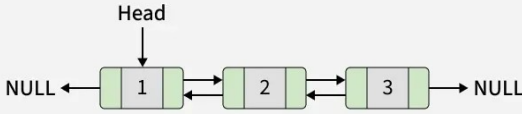


Doubly Linked List

A doubly linked list is a more complex data structure than a singly linked list, but it offers several advantages. The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions. This is because each node in the list contains a pointer to the previous node and a pointer to the next node. This allows for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of the list in both directions.



Representation of Doubly Linked List in Data Structure

In a data structure, a doubly linked list is represented using nodes that have three fields:

- ① Data
- ② A pointer to the next node (next)
- ③ A pointer to the previous node (prev)



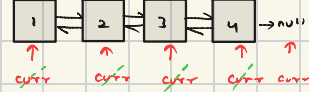
a) Traversal

a) Forward

```
private static void TraversalForward(DoublyLinkedList head){
    DoublyLinkedList curr = head;

    while (curr != null){
        System.out.print(curr.val + "⇌");
        curr = curr.next;
    }
    System.out.println("null");
}
```

#Example



b) Backward

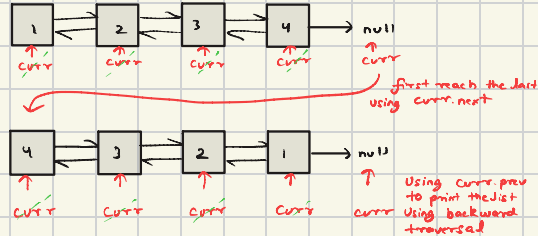
```
private static void TraversalBackward(DoublyLinkedList head) {
    DoublyLinkedList curr = head;

    while (curr.next != null) {
        curr = curr.next;
    }

    while (curr != null) {
        System.out.print(curr.val + "⇌");
        curr = curr.prev;
    }

    System.out.println("null");
}
```

#Example



i) Creation of Doubly Linked List

```
public class DoublyLinkedList {
    int val;
    DoublyLinkedList next;
    DoublyLinkedList prev;
    static DoublyLinkedList head;

    DoublyLinkedList() { }
    // default constructor

    DoublyLinkedList(int val) {
        this.val = val;
    }
    // constructor with value

    DoublyLinkedList(int val, DoublyLinkedList next, DoublyLinkedList prev) {
        this.val = val;
        this.next = next;
        this.prev = prev;
    }
    // constructor with next and prev node
}
```

ii) Insert At Start

```
private static DoublyLinkedList InsertAtStart(DoublyLinkedList node) {
    node.next = head;

    if (head != null) {
        head.prev = node;
    }

    head = node;

    return head;
}
```

4) Insert At Start

```
private static DoublyLinkedList InsertAtEnd(DoublyLinkedList node) {
    if (head == null) {
        head = node;
        return head;
    }

    DoublyLinkedList curr = head;

    while (curr.next != null) {
        curr = curr.next;
    }

    curr.next = node;
    node.prev = curr; → add this

    return head;
}
```

5) Insert At Position

```
private static DoublyLinkedList InsertAtPosition(DoublyLinkedList node, int position) {
    if (position < 1) {
        System.out.println("Position Not Found");
        return head;
    }

    if (position == 1) {
        node.next = head;
        if (head != null) {
            head.prev = node; → add this
        }

        head = node;
        return head;
    }

    DoublyLinkedList curr = head;
    int counter = 1;

    while (curr != null && counter < position - 1) {
        curr = curr.next;
        counter++;
    }

    if (curr != null) {
        node.next = curr.next;
        node.prev = curr; → add this

        if (curr.next != null) {
            curr.next.prev = node; } → add this

        curr.next = node;
    } else {
        System.out.println("Position Not Found");
    }

    return head;
}
```

6) Delete at Start

```
private static DoublyLinkedList DeleteAtStart() {
    if (head == null) return null;

    head = head.next;

    if (head != null) {
        head.prev = null; } → add this

    return head;
}
```

8) Delete At Position

```
private static DoublyLinkedList DeleteAtPosition(int position) {
    if (head == null) return null;

    if (position < 1) {
        System.out.println("Position Not Found");
        return head;
    }

    if (position == 1) {
        head = head.next;

        if (head != null) {
            head.prev = null; } → add this

        return head;
    }

    DoublyLinkedList curr = head;
    int counter = 1;

    while (curr != null && counter < position - 1) {
        curr = curr.next;
        counter++;
    }

    if (curr != null && curr.next != null) {
        curr.next.next.prev = curr; → add this
        curr.next = curr.next.next;
    } else {
        System.out.println("Position Not Found");
    }

    return head;
}
```

7) Delete At End

```
private static DoublyLinkedList DeleteAtEnd() {
    if (head == null) {
        return null;
    }

    if (head.next == null) {
        head = null;
        return head;
    }

    DoublyLinkedList curr = head;

    while (curr.next != null) {
        curr = curr.next;
    }

    curr.prev.next = null; → add this

    return head;
}
```

Why No Example for explanation? Because

almost all the operations has

similar visualization like

singly linked list is just a

diff. of (→) to (⇔)

And some examples have visuals.

9> Searching

```
private static boolean CheckIfPresent(int value){
    DoublyLinkedList curr = head;

    while (curr != null){
        if(curr.val == value){
            return true;
        }
        curr = curr.next;
    }

    return false;
}
```

10> Update Element

```
private static DoublyLinkedList UpdateElement(int oldValue , int newValue){
    DoublyLinkedList curr = head;

    while (curr != null){
        if(curr.val == oldValue){
            curr.val = newValue;
            break;
        }

        curr = curr.next;
    }

    return head;
}
```

11> Reverse

```
private static DoublyLinkedList ReverseList(DoublyLinkedList head) {
    DoublyLinkedList curr = head;
    DoublyLinkedList newHead = null;

    while (curr != null) {
        DoublyLinkedList temp = curr.next;

        curr.next = curr.prev;
        curr.prev = temp; → add this

        newHead = curr;
        curr = temp;
    }

    return newHead;
}
```